



Recall

Never miss another class, quiz, or deadline.
An AI study assistant built for University of Lahore students.

Muazzam Ali

BSCS, 6th Semester · University of Lahore

Live at recall-kohl-mu.vercel.app

Source files	93
Lines of application code	8,377
Database migrations	27
Automated checks	176
Running cost	PKR 0 / month

1. The problem

UOL students keep their academic life in three places that do not talk to each other, and the cost of that is measured in missed deadlines.

Where it lives	Why it fails
Slate (the university's Moodle)	Quiz and assignment deadlines sit in a calendar nobody opens.
WhatsApp groups	Timetables arrive as photos, then vanish into the scroll.
Nowhere	There are no reminders at all. You find out about a quiz an hour before it closes.

Recall unifies the three and, crucially, tells you before it matters rather than after.

2. What it does

Feature	What it solves
Slate sync	Reads quiz and assignment deadlines automatically. No copying, no manual entry.
Timetable import	Upload a photo or PDF of the timetable; a vision model reads the grid and you correct it.
Reminders	Web push 15 minutes before every class and 24h / 2h / 30min before every deadline.
Attendance	UOL detains students below 75%. Tracks each course and answers the only question that matters: can I miss
Study planner	Turns a topic list plus real deadlines into a schedule that works around actual classes.
Ask Recall	A tutor that already knows what is due and when you are free, so advice is specific rather than generic.

3. How it works

Every piece runs on a free tier. The interesting constraint is that Slate cannot be read by a server, which shapes the whole design.

Google sign-in no password is ever stored, and only @student.uol.edu.pk accounts are admitted

Browser extension fetches the Slate calendar from inside a real Slate page, then posts only its contents
the calendar link never leaves the computer

Timetable upload PDF or photo rendered to images, read by a vision model, corrected by the student, saved

Reminders pg_cron inside Postgres fires every 5 minutes
GitHub Actions is a second, independent trigger
Web Push delivers to phone and laptop alike

Design decisions worth defending

- **No portal passwords, ever.** Sign-in is Google OAuth and the Slate connection is a read-only calendar link. This is the difference between a tool a university could recommend and one it would ban.
- **No all-powerful database key.** The usual way to let a scheduled job read every user's rows is a key that bypasses row-level security. Recall uses narrow database functions instead, so no single leaked credential exposes everyone.
- **The AI is never trusted with scheduling.** The planner asks a model for a plan, then programmatically deletes any block that collides with a real class. A study session on top of a lecture would make the whole plan useless, so the rule is enforced in code, not in the prompt.
- **Extraction is a first draft.** Free vision models misalign columns on dense grids, so every extracted class is editable before it is saved. Wrong times mean wrong reminders, which is the one thing this app cannot get wrong.

4. Engineering challenges

These are the problems that took real work. Each was diagnosed by measurement rather than guesswork, and the figures below are the actual numbers observed.

CHALLENGE 1

Slate cannot be read by a server

The problem. Slate sits behind Cloudflare, which rejects server-to-server requests. A plain request for the calendar returns 403 no matter what headers are sent. Requests from an extension's own origin are refused too.

The fix. What Cloudflare does allow is a real page asking for its own data. A browser extension finds or briefly opens a Slate tab and runs the fetch inside that page, where it is a genuine same-origin request. This is not a way around a security control: it is the student's own browser fetching the student's own calendar, which is exactly what is meant to be allowed.

CHALLENGE 2

The AI provider was blocked in Pakistan

The problem. Google's Gemini API returned 403 from two separate cloud projects. The cause was regional: the developer API refuses requests from some countries, Pakistan included.

The fix. Routed every model call through OpenRouter, which proxies models from anywhere and offers a free tier. No paid service is used anywhere in the project.

CHALLENGE 3

Free AI models are unreliable, and fail silently

The problem. The study planner stopped working with no code change. Testing all 19 free models against the real prompt showed the entire model chain had died: two of the three timed out past 75 seconds every single run. One pool answered a question correctly and returned a payment error three minutes later.

The fix. Requests now race the whole chain at once and take the first usable answer, with a per-call timeout so one stalled pool cannot hold the request open. A diagnostic script re-measures which pools are alive. Free models also truncate mid-JSON constantly, so a repairing parser closes the object and keeps what survived rather than discarding the answer.

```
Before: every request timed out, then failed
After: a complete study plan in 3.5 seconds
```

CHALLENGE 4

A timetable grid loses its meaning as text

The problem. Extracting the text layer from a timetable PDF returned zero usable classes. Which day and period a class occupies is carried by its position in a grid, and reading the text throws that away.

The fix. PDFs are rasterised to images and read by a vision model, so the layout is actually seen. Accuracy is treated as a first draft: the student reviews and corrects every row before anything is saved.

CHALLENGE 5

The reminder scheduler was barely running

The problem. A student with four classes received one notification. The scheduling logic was correct; the job simply was not running. GitHub Actions was asked to run every ten minutes and its actual history told a different story.

The fix. Reminders moved into the database itself, where `pg_cron` fires every five minutes with nothing in between to deprioritise it. The GitHub workflow stayed as a second, independent trigger, and because every reminder is claimed in the database before it is sent, two triggers can never double-send. A reminder that arrives late is now sent anyway, worded honestly, because silence is the only outcome with no value.

```
Requested: every 10 minutes (144 runs a day)
26 August: about 30 runs
27 August: 3 runs 00:48, 10:36, 20:43
28 August: 1 run 04:29
```

```
After pg_cron: 09:05:00, 09:10:00, ... exactly on time
```

CHALLENGE 6

Syncing only worked when Slate was already open

The problem. The extension opens its own Slate tab in the background when none exists, and that path failed with 403 while a tab opened by hand worked perfectly.

The fix. Chrome throttles background tabs to roughly one timer tick per second, and Cloudflare's browser check depends on timers, so a hidden tab clears far more slowly than a visible one. The fetch was going out before clearance landed. The extension now waits until the page is genuinely Moodle rather than merely not a challenge, and retries once.

CHALLENGE 7

Half the screen turned black on mobile

The problem. Intermittently, on phones only, a black rectangle covered part of the page.

The fix. The dashboard kept a permanently animating 3D layer alive, wrapping an element with a backdrop blur, beside two large blur filters. WebKit re-tiles those layers as the animation runs and, under the memory pressure a phone reaches long before a laptop, hands back a tile it never painted. An unpainted tile is black. All of it is now disabled on touch screens, where none of that depth was visible at 390 pixels wide anyway.

CHALLENGE 8

A rate limit that counted every attempt and blocked none

The problem. A throttle was added to the device-pairing endpoint. Testing it with 35 wrong codes produced 35 rejections and a counter still reading zero.

The fix. The function incremented a counter and then raised an exception for the invalid path, and raising aborts the transaction, which rolls back the increment. It looked exactly like rate limiting from the outside and did nothing at all. The failure paths now return a value instead of raising, so the count survives. This is recorded here because it is the most useful lesson of the project: a security control that is never tested is a control that may not exist.

```
Before: 45 wrong codes -> 45 rejected, 0 throttled  
After: 45 wrong codes -> 40 rejected, 5 throttled
```

5. Security

The application was audited end to end and every finding was fixed and verified against the live system rather than assumed.

Finding	Resolution
Open redirect in sign-in	A crafted link produced a URL where the real domain was parsed as a username, sending the student to a
Inline scripts permitted	The content security policy allowed any inline script, which is the exact attack it exists to stop. Replaced w
No response hardening	The app could be framed by any site, which matters because one click inside it signs you out. Added clickj
Weak pairing codes	Eight characters, roughly 4 billion combinations. Now twelve, roughly 280 trillion.
No rate limiting anywhere	One student could exhaust the shared AI quota for the whole university. Limits now live in the database, b
Permanent device credential	Sync tokens never expired and could not be revoked. They now expire when unused, every account can r
Unbounded calendar import	One crafted calendar could insert roughly ten thousand rows and exhaust the free-tier database. Capped,
Unrestricted calendar link	The extension fetched whatever address it was given from inside a logged-in Slate page, including addres

Two properties are proven rather than assumed. The public database key ships in every browser by design, so its inability to read anything is tested against the live database. And student A being unable to read student B's data is demonstrated by impersonating a real signed-in session in Postgres.

```
victim owns: 14 classes, 4 devices, 3 push subscriptions
attacker sees: 0 0 0
attacker's own: 12 deadlines, 11 classes (the session works)

update victim's rows -> 0 insert as victim -> refused
```

6. Testing

176 automated checks run from one command, plus live checks against the production database. The scheduling rules and the attendance arithmetic are pure functions specifically so they can be tested without a database, a browser or a phone.

What is checked	Why it earns its place
Reminder scheduling	Silent missed reminders are this project's worst failure mode.
Attendance arithmetic	Off by one here is the difference between sitting an exam and being detained, so every boundary is as
Calendar parsing	Real Moodle output, including the quirks that only appear in practice.
Loose JSON repair	Free models truncate constantly; a partial plan beats an error message.
Sign-in redirect	Seven attack strings, asserting the host can never change.
Content security policy	A policy rots quietly; the first assertion is that inline scripts are still forbidden.
Extension integrity	Written after a refactor left a function being called that no longer existed - an error that would only have
Cross-user isolation	The single most important property the application has.

7. Stack and cost

Layer	Choice	Cost
Framework	Next.js 16, App Router, Server Actions	Free
Database	Supabase Postgres, row-level security on every table	Free tier
Auth	Supabase Auth with Google OAuth, UOL accounts only	Free
AI	OpenRouter free models, raced for availability	Free
Notifications	Web Push with VAPID, no third-party service	Free
Scheduling	pg_cron in Postgres, GitHub Actions as backup	Free
Hosting	Vercel	Free tier

Total running cost is zero. This was a design constraint from the start, not an accident, and it is what makes the app something every student can use rather than something that needs funding to survive.

8. Status and what comes next

Working today. Sign-in restricted to UOL accounts, Slate sync, timetable import, dashboard, reminders on phone and laptop, attendance tracking, study planner, and Ask Recall. Deployed and in daily use.

The most valuable next step is not code. Every feature that touches Slate depends on a browser extension on a laptop, purely because Cloudflare will not accept a server. If the university allowed Recall's server to read the calendar export endpoint, the extension would disappear entirely and the app would work identically on any phone, with no installation at all. That is a permission question rather than an engineering one, and it is the single biggest improvement available.

Built by Muazzam Ali, a University of Lahore student, for University of Lahore students.